

Report for Assignment 1

Mauritz Almgren¹ `manz24@student.bth.se`

Blekinge Tekniska Högskola Valhallavägen 10, 371 79 Karlskrona, Sweden
`info@bth.se`

1 Introduction

This report discusses the implementation of the reflex-based and utility-based agents from Problem 1 and the implementation and performance of Breadth-First Search (BFS) and Depth-First Search (DFS) for the maze problem in Problem 2.

2 Problem 1: Shopping Assistant Agents

The reflex agent was implemented using if-then rules. It checks each item and buys it if the purchase remains within the budget and either the price is at most five or the satisfaction is greater than eight.

The utility agent evaluates all possible combinations within the budget. It maximizes total satisfaction and uses total price as a tie-breaker, selecting the cheaper combination when two combinations have equal satisfaction. The utility agent therefore makes decisions based on the overall utility of a combination, while the reflex agent makes decisions based only on predefined rules.

3 Problem 2: Maze Solving

The maze was represented as a 2D graph, where 1 represents walls and 0 represents walkable nodes. BFS was implemented using a queue and DFS using a stack. A parent mapping was used by both algorithms to reconstruct the path from the goal back to the starting node. The implementation also records the number of visited nodes and the maximum search structure size.

BFS found a path of 26 steps after visiting 80 nodes, with a maximum frontier size of 6. DFS also found a 26-step path after visiting 80 nodes, with a maximum stack depth of 11. Although both algorithms found paths of equal length in this maze, they followed different paths.

BFS guarantees the shortest path in an unweighted graph, making it well suited for this maze. Its main weakness is that its memory usage can increase substantially because it stores the frontier of nodes waiting to be explored. DFS generally uses less search-state memory by following one branch at a time, but it does not guarantee a shortest path and may spend time exploring an unsuitable branch before backtracking.

For this particular maze, both algorithms produced the same path length and visited the same number of nodes. However, BFS is the more suitable algorithm for the problem because finding the shortest path is required and the maze is an unweighted graph. DFS can find a solution, but it cannot guarantee that the solution is optimal.